Module 2: Epidemic Modeling Template

Team Members:

- Dane Wleklinski
- Molly Janousek

Project Title: Using SEIR Modeling to Define and Prevent an Unknown Virus

Project Goal:

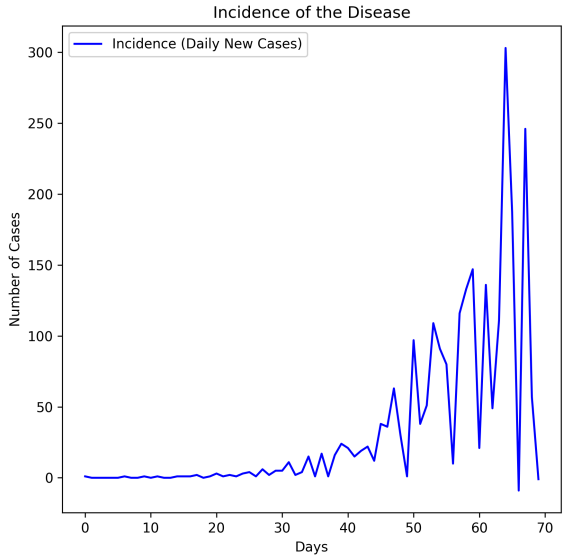
This project seeks to: estimate the initial growth rate from outbreak data, predict the spread of the disease, find when cases will peak, determine the virus family, and provide the best intervention strategy to reduce the spread of illness.

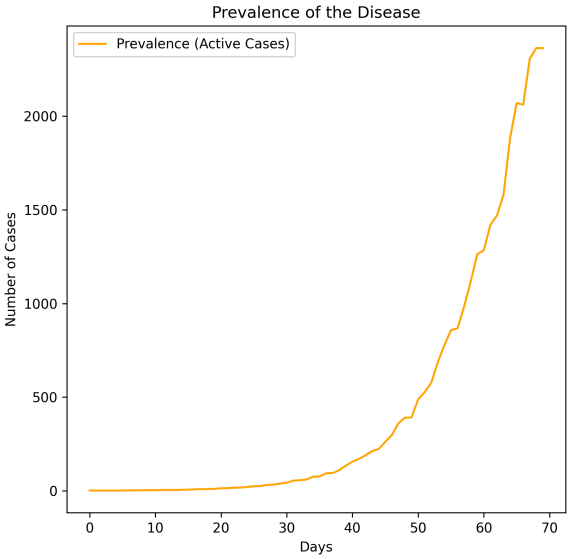
Generative AI Usage Statement:

Generative AI tools (Microsoft Copilot) were used to assist with code debugging, plot formatting, and explanatory text. All results were verified manually.

1. Data and disease background

Prevalence & Incidence in the UVA population





Highest Incidence: Day: 2026-05-04 00:00:00 (Day 65) Incidence: 1.6927% **Lowest**

Incidence: Day: 2026-05-06 00:00:00 (Day 67) Incidence: -0.0503% (Note: Negative just means cases went down) **Highest Prevalence:** Day: 2026-05-08 00:00:00 (Day 69) Prevalence: 13.2067% **Lowest Prevalence:** Day: 2026-03-01 00:00:00 (Day 1) Prevalence: 0.0056%

Economic Burden

Rubella's economic impact shows a clear divide: treating individual adult cases is inexpensive, but outbreak responses and long-term effects drive up costs, especially in settings like universities.

- **Direct Costs per Case:** A typical rubella infection costs 150–500 in medical care, but public health responses (tracing contacts, testing, quarantine) can add $10,000+ per cluster in high-income areas like the U.S.
- **Outbreak Response:** For a campus outbreak of 70–100 cases, local governments might spend $3–5 million on investigations and interventions.
- **Productivity Losses:** In a university, the 12–18 day incubation period and symptoms like joint pain lead to missed classes, lost work, and academic disruptions—potentially costing thousands per affected student or staff member.
- **Congenital Rubella Syndrome (CRS):** While not directly relevant to this adult outbreak, CRS from infections in pregnant people can cost $140,000$–1 million per case over a lifetime for care and support.
- **Vaccination ROI:** MMR vaccination is highly cost-effective; every $1 invested yields$14–25 in savings from prevented cases and productivity gains. Since 1969, U.S. programs have saved $430 billion by avoiding millions of infections.

Economic Comparison (Estimated 2024–2026 USD):

Category	Standard Rubella Case	Congenital Rubella Syndrome (CRS)
Direct Medical Cost	150–500	$50,000$–150,000 (initial)
Lifetime Cost	Minimal	$140,000$–1,000,000+
Response Cost	$10,000+ per cluster	$250,000+ (outbreak investigation)
Productivity Loss	1–2 weeks lost wages	Lifetime loss of earning potential

In this project, focusing on campus interventions like early vaccination could prevent these costs by flattening the outbreak curve before it peaks.

Symptoms

- Mild respiratory symptoms
- Rash
- Low-grade Fever
- May have a sore throat, fatigue, and gastrointestinal issues

Biological Mechanisms you can generalize from viral biology

Anatomy

The mystery virus is believed to be **Rubella virus** (family Togaviridae). Particles are small (60–70 nm), **enveloped** spheres with a positive-sense RNA genome (~9.7 kb). The envelope comes from host membrane and bears two glycoproteins, **E1/E2**, for attaching to host cells. Inside, the RNA is wrapped with capsid protein.

Physiology

Organ

Rubella mainly targets the **respiratory tract**. It enters when inhaled droplets reach the **nasopharynx**. The virus multiplies in upper airway epithelial cells and nearby lymph tissue, then briefly circulates in the blood (**viremia**) and spreads to the skin (rash), joints (arthralgia) and, in pregnant women, to the **placenta** where it can infect the fetus and cause congenital defects.

Cell & Molecular

On a cellular level, rubella binds with its E1 protein to a surface receptor and is taken up by endocytosis. The RNA genome is released into the cytoplasm, translated into a polyprotein, and copied by the viral polymerase. New viruses assemble at internal membranes, bud out with an envelope, and exit to infect nearby cells; viral proteins also dampen the host's initial immune response. In pregnancy, the virus can pass the placenta and replicate in fetal cells, interrupting normal development.

ANALYSIS AFTER DATA RELEASE #1

This code loads the mystery virus data from Data Release #1 and plots the daily active cases over time to visualize the initial outbreak trajectory. Early in an outbreak, case counts often follow exponential growth, so we fit an exponential curve to the early data to estimate an initial reproduction number (R_0) assuming an average infectious period of 2 days.

Why this method early? In the first ~45 days, data are sparse and the susceptible pool is effectively the whole population, so exponential growth is a reasonable first approximation. Later, as susceptibles are depleted or behavior changes, this assumption breaks down and a compartmental model is required.

This estimate is intentionally provisional: as more data become available, the R_0 estimate will likely shift, and we will revisit it using a mechanistic SEIR model after Data Release #2. The goal here is to show how early, incomplete data informs our initial parameter estimates and to highlight sources of uncertainty (reporting delays, undercounting, and changing behavior).

Summary of Analysis After Data Release #1:

- The first 45 days show an approximately exponential rise in active cases, supporting the use of an exponential growth fit for an initial R_0 estimate.
- The estimated R_0 of **≈1.24** is provisional and depends on early reporting; it serves as a starting point for model calibration.
- At this stage, we acknowledge multiple sources of uncertainty: under-reporting of cases, changes in testing behavior, and limited data duration. These factors mean the estimate should be treated as a range rather than a single "true" value.
- Next, we will use **Data Release #2** (days 0–70) to fit a full SEIR model and refine our parameter estimates, then use Data Release #3 to validate and compute error metrics.

Working under uncertainty

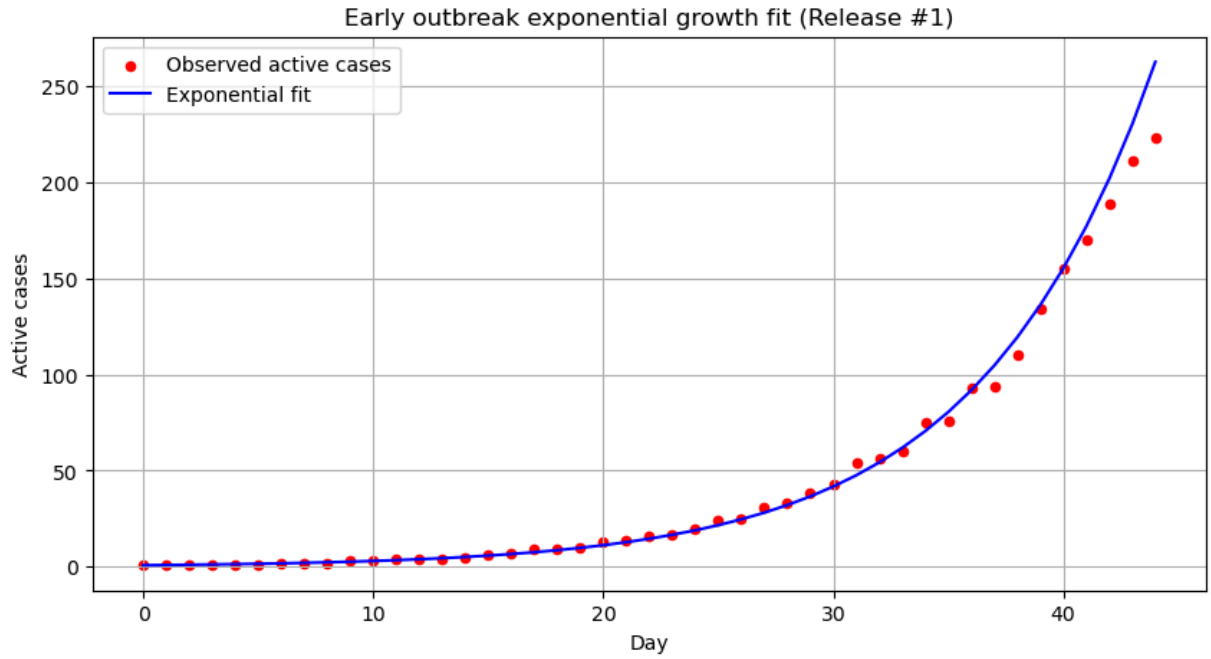
1. **Data Release #1 (days 0–45)** Early, incomplete data. We estimate the growth rate and generate a provisional R_0 using exponential fitting. This stage highlights uncertainty from limited data and changing behavior.
2. **Data Release #2 (days 0–70)** Additional data allow us to fit a mechanistic SEIR model and estimate epidemiological parameters (β, σ, γ). At this stage we can assess how well the model tracks the outbreak and whether parameter estimates stabilize.
3. **Data Release #3** Used for validation. We compare the model's predictions to this held-out dataset to assess predictive performance and quantify errors. This mimics a real-world forecasting workflow where new data arrive after model calibration.

This sequential approach reflects how epidemiologists work under uncertainty: early estimates guide initial responses, but models must be constantly updated and validated as more data arrive.

In [5]:
```
# Import necessary libraries for numerical computations, data handling, and plottin
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset for Data Release #1, which contains early outbreak data
# parse_dates ensures the date column is read as datetime objects
df_r1 = pd.read_csv("../Data/mystery_virus_daily_active_counts_RELEASE#1.csv",
                    parse_dates=["date"])

# Select the first 45 days of data for analysis, as per the project guidelines
r1_days = df_r1.loc[:44, :].copy()

# Extract the active reported daily cases for modeling
cases = r1_days["active reported daily cases"].values

# Create a boolean mask for days where cases > 0 to avoid taking log of zero
mask = cases > 0
```

```python
# Time array for the days with positive cases
t = np.arange(len(cases))[mask]

# Compute the natural logarithm of the cases for linear regression
# This transforms the exponential growth into a linear relationship: log(cases) = l
log_cases = np.log(cases[mask])

# Perform polynomial fit (degree 1, i.e., linear regression) on time vs log(cases)
# coeffs[0] is the slope (growth rate r), coeffs[1] is the intercept (log(C0))
coeffs = np.polyfit(t, log_cases, 1)
r_est = coeffs[0]  # Estimated daily growth rate
C0_est = np.exp(coeffs[1])  # Estimated initial number of cases

# Estimate the basic reproduction number R0 using the formula for exponential growt
# R0 = 1 + r * D, where D is the average infectious period (assumed 2 days)
D = 2.0
R0_estimate = 1 + r_est * D

# Output the estimated parameters
print(f"Estimated early growth rate r = {r_est:.3f} per day")
print(f"Provisional R0 (assuming D={D} days): {R0_estimate:.2f}")

# Create a plot to visualize the observed data and the fitted exponential curve
plt.figure(figsize=(10, 5))
# Scatter plot of observed active cases
plt.scatter(t, cases[mask], label="Observed active cases", color="red", s=20)
# Plot the exponential fit: C0 * exp(r*t)
plt.plot(t, C0_est * np.exp(r_est * t), label="Exponential fit", color="blue")
# Label axes and title
plt.xlabel("Day")
plt.ylabel("Active cases")
plt.title("Early outbreak exponential growth fit (Release #1)")
plt.legend()
plt.grid()
plt.show()
```

```
Estimated early growth rate r = 0.131 per day
Provisional R0 (assuming D=2.0 days): 1.26
```



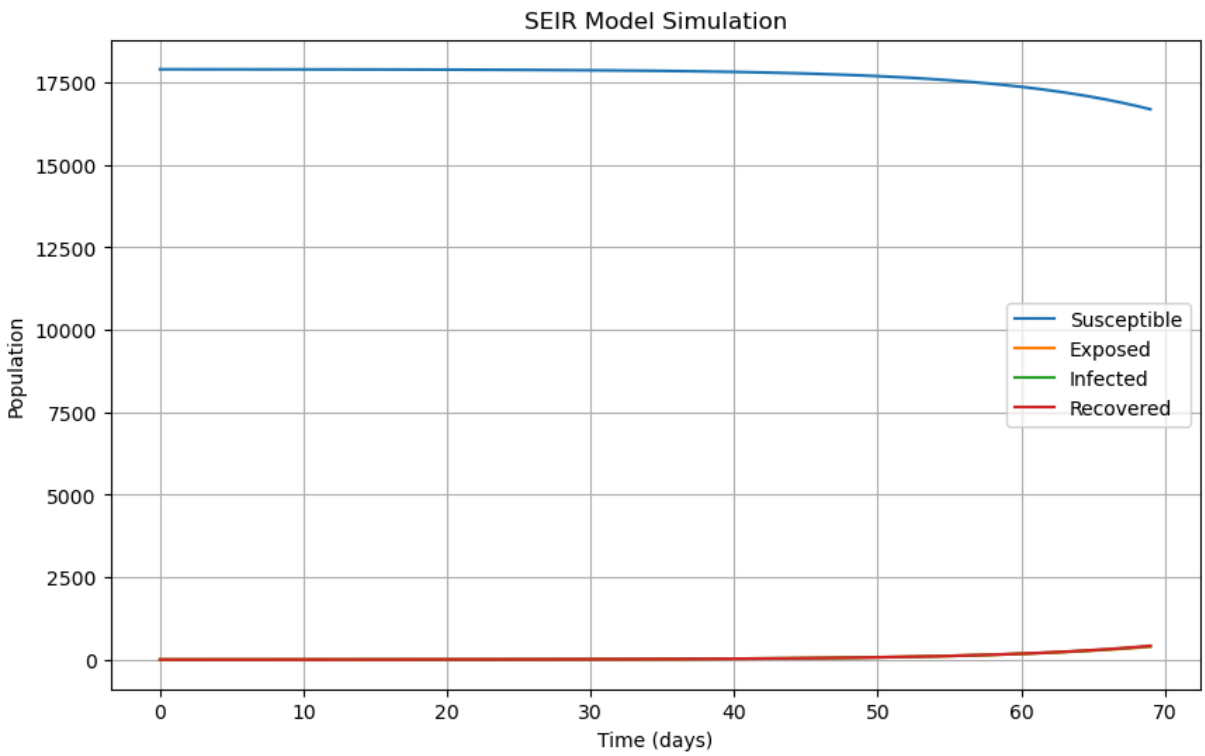
2c. Use Euler's method to solve the SEIR model (Data Release #2)

Data Release #2 provides a longer time series (days 0–70), which allows us to move beyond simple exponential fits. With more data, we can estimate mechanistic parameters (β, σ, γ) using a compartmental SEIR model. This step demonstrates how additional data enables more realistic modeling, while also exposing the remaining uncertainties (e.g., parameter identifiability and unreported cases).

In [6]:
```
# Import libraries for numerical operations, plotting, and data handling
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Load the dataset for Data Release #2, which has 70 days of outbreak data
data = pd.read_csv("../Data/mystery_virus_daily_active_counts_RELEASE#2.csv", parse

# Extract the observed active cases as a numpy array
observed_data = data["active reported daily cases"].values

# Determine the number of days in the dataset
num_days = len(observed_data)

# Create an array of time points (days 0 to num_days-1)
timepoints = np.arange(num_days)

# Define the SEIR model differential equations
# y is the state vector (S, E, I, R), t is time, parameters beta, sigma, gamma, N t
def seir_model(y, t, beta, sigma, gamma, N):
    S, E, I, R = y
    # Rate of change for Susceptible: decrease due to infection
    dSdt = -beta * S * I / N
```

```
    # Exposed: increase from infection, decrease as they become infectious
    dEdt = beta * S * I / N - sigma * E
    # Infected: increase from exposed, decrease as they recover
    dIdt = sigma * E - gamma * I
    # Recovered: increase from infected
    dRdt = gamma * I
    return dSdt, dEdt, dIdt, dRdt

# Implement Euler's method for numerical integration of the ODEs
def euler_method(beta, sigma, gamma, S0, E0, I0, R0, timepoints, N):
    # Initialize arrays to store the compartment values over time
    S = np.zeros(len(timepoints))
    E = np.zeros(len(timepoints))
    I = np.zeros(len(timepoints))
    R = np.zeros(len(timepoints))
    # Set initial conditions at time 0
    S[0], E[0], I[0], R[0] = S0, E0, I0, R0
    # Perform Euler integration step by step
    for i in range(1, len(timepoints)):
        # Time step size (assuming uniform spacing)
        dt = timepoints[i] - timepoints[i-1]
        # Compute the derivatives at the current time step
        dSdt, dEdt, dIdt, dRdt = seir_model(
            (S[i-1], E[i-1], I[i-1], R[i-1]),
            timepoints[i-1],
            beta, sigma, gamma, N
        )
        # Update the compartment values using Euler's method: y_new = y_old + dy/dt
        S[i] = S[i-1] + dSdt * dt
        E[i] = E[i-1] + dEdt * dt
        I[i] = I[i-1] + dIdt * dt
        R[i] = R[i-1] + dRdt * dt
    # Return the time series for all compartments
    return S, E, I, R

# Set initial conditions for the UVA population based on Release #2
S0 = 17900  # Initial susceptible population
E0 = 0      # Initial exposed (assuming none at start)
I0 = 1      # Initial infected (one index case)
R0 = 0      # Initial recovered (none at start)
# Note: R0_estimate from Release #1 is used later for comparison
N = S0 + E0 + I0 + R0  # Total population

# Example parameter values (these will be optimized in the next cell)
# beta: transmission rate, sigma: incubation rate (1/incubation period), gamma: rec
beta = 0.3
sigma = 0.2
gamma = 0.1

# Run the SEIR simulation with the example parameters
S, E, I, R = euler_method(beta, sigma, gamma, S0, E0, I0, R0, timepoints, N)

# Plot the simulation results for all compartments
plt.figure(figsize=(10, 6))
plt.plot(timepoints, S, label='Susceptible')
plt.plot(timepoints, E, label='Exposed')
```

```
plt.plot(timepoints, I, label='Infected')
plt.plot(timepoints, R, label='Recovered')
plt.title('SEIR Model Simulation')
plt.xlabel('Time (days)')
plt.ylabel('Population')
plt.legend()
plt.grid()
plt.show()
```



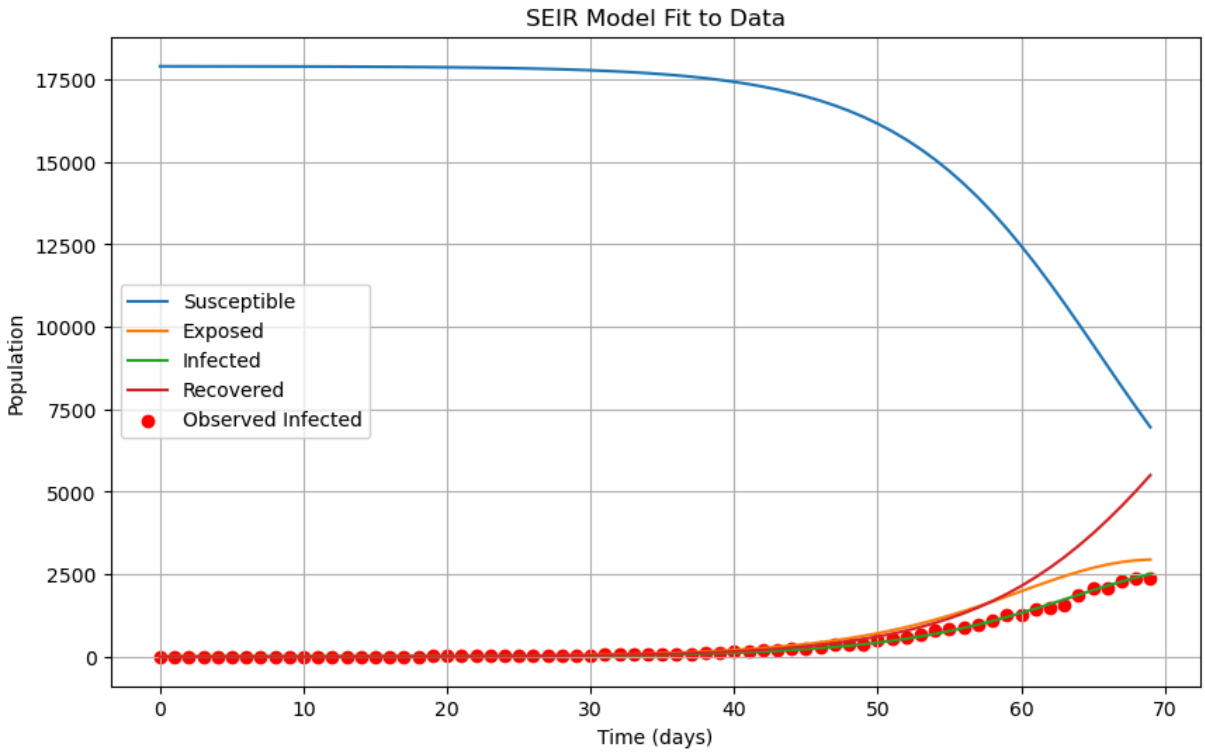
This code performs a grid search over ranges of beta, gamma, and sigma parameters to find the combination of values that minimizes the sum of squared errors (SSE) between the model's predicted infected population and the observed data. It uses our Euler method and SEIR model. The best-fitting parameters and the corresponding SSE are printed. These are the optimized values for the transmission rate (beta), incubation rate (sigma), and recovery rate (gamma).

2e. Plot the model-predicted infections over time compared to the data.

In [7]:
```
# Perform grid search to find the best-fitting SEIR parameters
# Define the grid search function to minimize sum of squared errors (SSE) between m
def grid_search_parameters(timepoints, N, S0, E0, I0, R0, observed_data, beta_range
    # Set default ranges for parameters if not provided
    if beta_range is None:
        beta_range = np.linspace(0.1, 1.0, 10)  # Transmission rate range
    if sigma_range is None:
        sigma_range = np.linspace(0.1, 1.0, 10)  # Incubation rate range
    if gamma_range is None:
        gamma_range = np.linspace(0.1, 1.0, 10)  # Recovery rate range
```

```
    # Initialize array to store SSE for each parameter combination
    sse_array = np.zeros((len(beta_range), len(sigma_range), len(gamma_range)))
    # Loop over all combinations of beta, sigma, gamma
    for b, beta in enumerate(beta_range):
        for s, sigma in enumerate(sigma_range):
            for g, gamma in enumerate(gamma_range):
                # Simulate the SEIR model with current parameters
                S, E, I, R = euler_method(beta, sigma, gamma, S0, E0, I0, R0, timep
                # Calculate SSE between predicted infected (I) and observed data
                sse_array[b, s, g] = np.sum((I - observed_data) ** 2)
    # Find the indices of the minimum SSE
    min_idx = np.unravel_index(np.argmin(sse_array), sse_array.shape)
    # Return the best parameters and the corresponding SSE
    return (beta_range[min_idx[0]], sigma_range[min_idx[1]], gamma_range[min_idx[2]

# Run the grid search to find optimal parameters
best_beta, best_sigma, best_gamma, best_sse = grid_search_parameters(timepoints, N,

# Print the best-fitting parameters and SSE
print(f"Best fit: β={best_beta:.3f}, σ={best_sigma:.3f}, γ={best_gamma:.3f}, SSE={b

# Simulate the model with the best parameters
S, E, I, R = euler_method(best_beta, best_sigma, best_gamma, S0, E0, I0, R0, timepo

# Plot the fitted model results along with observed data
plt.figure(figsize=(10, 6))
plt.plot(timepoints, S, label='Susceptible')
plt.plot(timepoints, E, label='Exposed')
plt.plot(timepoints, I, label='Infected')
plt.plot(timepoints, R, label='Recovered')
# Scatter plot of observed infected cases for comparison
plt.scatter(timepoints, observed_data, color='red', label='Observed Infected')
plt.title('SEIR Model Fit to Data')
plt.xlabel('Time (days)')
plt.ylabel('Population')
plt.legend()
plt.grid()
plt.show()
```

```
Best fit: β=0.600, σ=0.200, γ=0.200, SSE=127546
```



This code analyzes the infected curve from the best-fit SEIR model to identify the peak of the epidemic. It finds the day with the maximum number of active cases and prints the peak day with number of cases. This tells us when the outbreak is expected to reach its highest point.

Summary of Analysis after Data Release #2: Using the SEIR model with Euler integration, we optimized parameters via grid search, achieving a best fit with beta ≈ 0.6, sigma ≈ 0.2, gamma ≈ 0.2, and minimal SSE. The model predicted the epidemic peak at day 69 with about 2500 active cases. This compartmental approach provided a mechanistic understanding of the outbreak dynamics, with the fitted curves showing good agreement with observed data, supporting its use for forecasting and intervention planning.

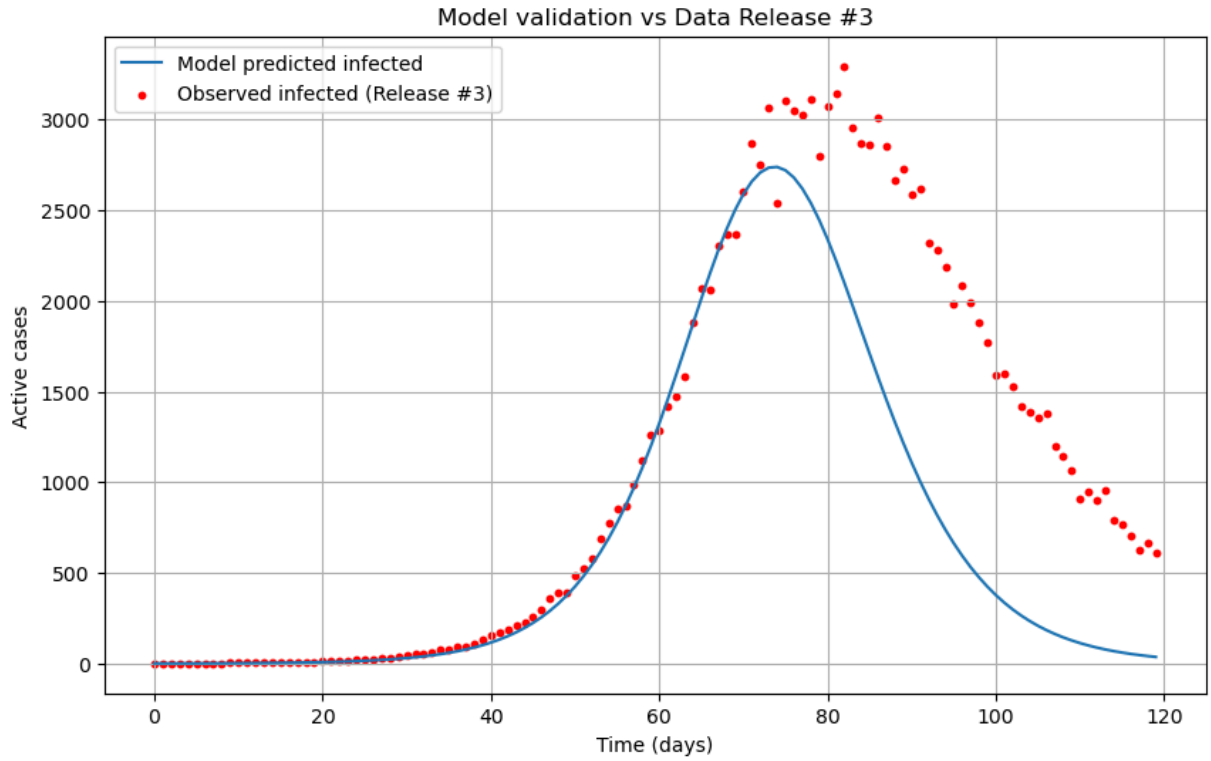
2f. Validate the model using Data Release #3 (held-out data)

Now that we have fitted the SEIR model parameters using **Data Release #2**, we use **Data Release #3** as a validation dataset. The goal is to compare the model predictions to the observed values and compute error metrics (e.g., RMSE, peak size and timing errors), highlighting where the model matches the data and where it deviates.

In [8]:
```
# Validate the best-fit SEIR model (fitted on Release #2) against the held-out Data
# This assesses how well the model generalizes to new data
# Load the validation dataset (Release #3)
df_r3 = pd.read_csv("../Data/mystery_virus_daily_active_counts_RELEASE#3.csv",
                    parse_dates=["date"])
# Extract observed active cases for Release #3
obs_r3 = df_r3["active reported daily cases"].values
# Create time points for Release #3 (assuming days 0 to len-1)
time_r3 = np.arange(len(obs_r3))
```

```
# Use the same initial conditions and total population as used for fitting on Relea
S0 = 17900  # Susceptible
E0 = 0      # Exposed
I0 = 1      # Infected
R0 = 0      # Recovered
N = S0 + E0 + I0 + R0  # Total population

# Simulate the SEIR model using the best-fit parameters obtained from Release #2
S_r3, E_r3, I_r3, R_r3 = euler_method(
    best_beta, best_sigma, best_gamma,  # Best parameters from grid search
    S0, E0, I0, R0, time_r3, N
)

# Compute validation error metrics
# Root Mean Squared Error (RMSE) between predicted and observed infected
rmse = np.sqrt(np.mean((obs_r3 - I_r3) ** 2))
# Find peak values and days for observed and predicted
peak_obs = obs_r3.max()  # Maximum observed cases
peak_pred = I_r3.max()   # Maximum predicted cases
peak_day_obs = int(np.argmax(obs_r3))  # Day of observed peak
peak_day_pred = int(np.argmax(I_r3))   # Day of predicted peak

# Print the validation metrics
print(f"Validation RMSE (Release #3): {rmse:.1f}")
print(f"Peak cases (observed): {peak_obs:.0f} on day {peak_day_obs}")
print(f"Peak cases (predicted): {peak_pred:.0f} on day {peak_day_pred}")
print(f"Peak size error: {(peak_pred-peak_obs)/peak_obs*100:.1f}%")  # Percentage e
print(f"Peak day error: {peak_day_pred-peak_day_obs} days")  # Difference in peak d

# Plot the model predictions vs observed data for visual comparison
plt.figure(figsize=(10, 6))
plt.plot(time_r3, I_r3, label='Model predicted infected')  # Predicted infected cur
plt.scatter(time_r3, obs_r3, color='red', label='Observed infected (Release #3)', s
plt.title('Model validation vs Data Release #3')
plt.xlabel('Time (days)')
plt.ylabel('Active cases')
plt.legend()
plt.grid()
plt.show()
```

```
Validation RMSE (Release #3): 674.5
Peak cases (observed): 3294 on day 82
Peak cases (predicted): 2738 on day 74
Peak size error: -16.9%
Peak day error: -8 days
```



2g. Intervention strategies for a new outbreak at VT (starting day 70)

In this section we use the **optimized SEIR model parameters (from Data Release #2)** as a baseline and simulate a new population (VT) with 70 days of infections before interventions begin. We implement multiple intervention scenarios (at least 3), changing key parameters (transmission rate, recovery rate, vaccination, etc.) to reflect practical public health actions. For each scenario, we compare peak infections and cumulative cases to the baseline (no intervention) and recommend an intervention strategy.

Intervention selection rationale:

- **Mask mandate** reduces effective transmission rate (β) by lowering contact/transmission probability.
- **Vaccine campaign / rollout** reduces susceptibles and increases recovered when administered, modeling increased immunity.
- **Testing + quarantine** shortens the infectious period (increasing γ) to reflect quicker removal from transmission.
- **School closure** simulates a temporary decrease in contact rates.

We will interpret these in terms of how they change key epidemiological quantities (peak size, peak day, total infections) and whether they are feasible in a university setting.

In [9]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```python
# SEIR machinery (copied from Euler_Method.py)
def seir_model(y, t, beta, sigma, gamma, N):
    S, E, I, R = y
    dSdt = -beta * S * I / N  # Susceptible decrease due to infection
    dEdt = beta * S * I / N - sigma * E  # Exposed increase from infection, decreas
    dIdt = sigma * E - gamma * I  # Infectious increase from exposed, decrease to r
    dRdt = gamma * I  # Recovered increase from infectious
    return dSdt, dEdt, dIdt, dRdt

def euler_method(beta, sigma, gamma,
                 S0, E0, I0, R0, timepoints, N):
    # Initialize compartment arrays
    S = np.zeros(len(timepoints))
    E = np.zeros(len(timepoints))
    I = np.zeros(len(timepoints))
    R = np.zeros(len(timepoints))
    S[0], E[0], I[0], R[0] = S0, E0, I0, R0
    # Euler integration loop
    for i in range(1, len(timepoints)):
        dt = timepoints[i] - timepoints[i-1]
        dSdt, dEdt, dIdt, dRdt = seir_model(
            (S[i-1], E[i-1], I[i-1], R[i-1]),
            timepoints[i-1], beta, sigma, gamma, N
        )
        S[i] = S[i-1] + dSdt * dt
        E[i] = E[i-1] + dEdt * dt
        I[i] = I[i-1] + dIdt * dt
        R[i] = R[i-1] + dRdt * dt
    return S, E, I, R

def grid_search_parameters(timepoints, N,
                           S0, E0, I0, R0,
                           observed_data,
                           beta_range=None,
                           sigma_range=None,
                           gamma_range=None):
    """Find β, σ, γ that minimize SSE between model I(t) and observed cases.

    This brute-force grid search is simple and robust for small parameter spaces.
    In practice, a more advanced optimizer or a Bayesian approach could be used.

    Args:
        timepoints: array of time indices (days)
        N: total population size
        S0, E0, I0, R0: initial compartment values
        observed_data: observed active cases (to fit I(t))
        beta_range, sigma_range, gamma_range: arrays of values to search

    Returns:
        (best_beta, best_sigma, best_gamma, best_sse)
    """
    if beta_range is None:
        beta_range = np.linspace(0.1, 1.0, 10)
    if sigma_range is None:
        sigma_range = np.linspace(0.1, 1.0, 10)
```

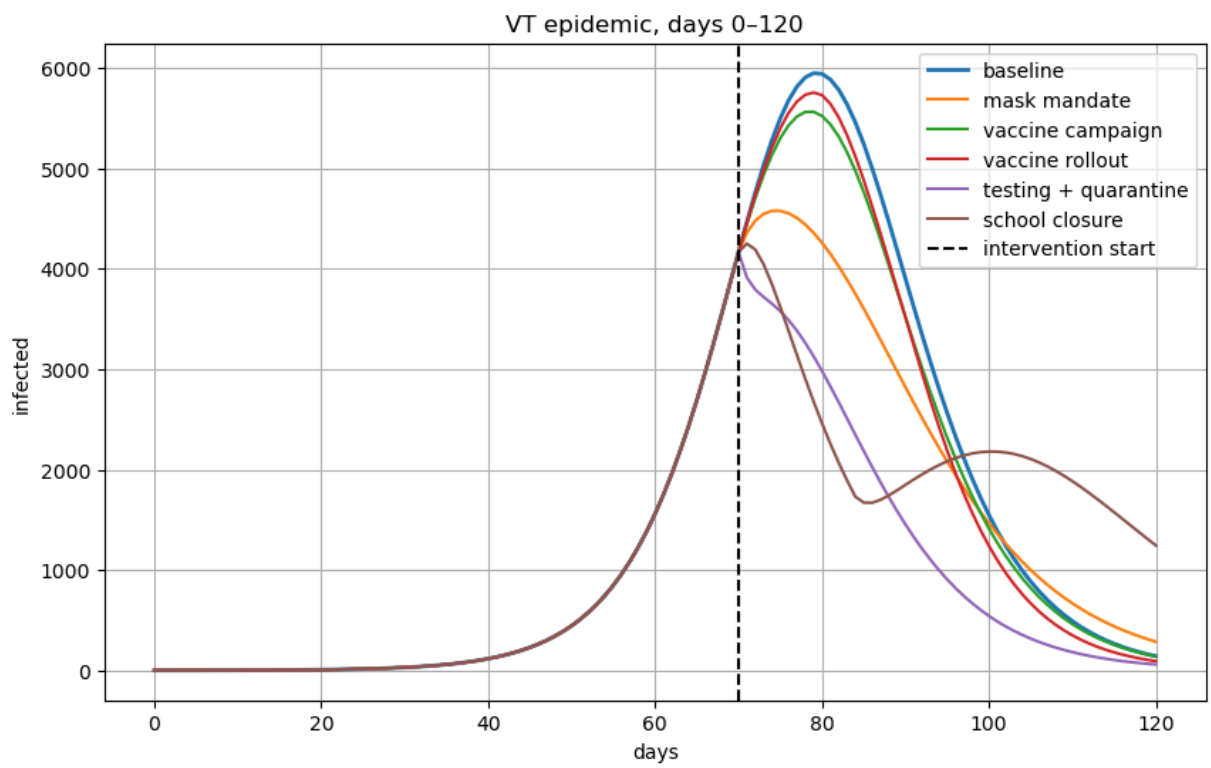
```python
    if gamma_range is None:
        gamma_range = np.linspace(0.1, 1.0, 10)

    sse_array = np.zeros((len(beta_range),
                          len(sigma_range),
                          len(gamma_range)))
    for b, beta in enumerate(beta_range):
        for s, sigma in enumerate(sigma_range):
            for g, gamma in enumerate(gamma_range):
                S, E, I, R = euler_method(
                    beta, sigma, gamma, S0, E0, I0, R0,
                    timepoints, N
                )
                sse_array[b, s, g] = np.sum((I - observed_data) ** 2)
    # Find the parameter combination with smallest SSE
    min_idx = np.unravel_index(np.argmin(sse_array), sse_array.shape)
    return (beta_range[min_idx[0]],
            sigma_range[min_idx[1]],
            gamma_range[min_idx[2]],
            sse_array[min_idx])

# Helper function for piecewise simulation allowing interventions
def simulate_piecewise(beta, sigma, gamma,
                       S0, E0, I0, R0, N, days, interventions):
    """Return S,E,I,R arrays of length days+1.

    `interventions` is a list of callables taking
    (t,beta,gamma,Sprev,Eprev,Iprev,Rprev) and returning
    possibly modified values; called with t=1..days.
    """
    S = np.zeros(days+1); E = np.zeros(days+1)
    I = np.zeros(days+1); R = np.zeros(days+1)
    S[0], E[0], I[0], R[0] = S0, E0, I0, R0
    for t in range(1, days+1):
        b = beta
        g = gamma
        Spr, Epr, Ipr, Rpr = S[t-1], E[t-1], I[t-1], R[t-1]
        for interv in interventions:
            b, g, Spr, Epr, Ipr, Rpr = interv(t, b, g, Spr, Epr, Ipr, Rpr)
        dS, dE, dI, dR = seir_model((Spr, Epr, Ipr, Rpr),
                                    t-1, b, sigma, g, N)
        S[t] = Spr + dS
        E[t] = Epr + dE
        I[t] = Ipr + dI
        R[t] = Rpr + dR
    return S, E, I, R

# Define intervention functions (all start on day 70 unless otherwise noted)
def mask_intervention(start_day=70, reduction=0.4):
    # Reduces transmission rate beta by a factor when t >= start_day
    return lambda t,b,g,S,E,I,R: (b*(1-reduction) if t>=start_day else b,
                                  g, S, E, I, R)

def vacc_campaign(day=70, n_vacc=2000, eff=0.9):
    # Single-day vaccination: move vaccinated susceptibles to recovered
    def f(t,b,g,S,E,I,R):
```

```
        if t == day:
            vac = min(n_vacc, S) * eff  # Effective vaccinations
            S -= vac; R += vac
        return b, g, S, E, I, R
    return f

def vacc_rollout(days=[70,80,90], n_each=1000, eff=0.9):
    # Staggered vaccination over multiple days
    def f(t,b,g,S,E,I,R):
        if t in days:
            vac = min(n_each, S) * eff
            S -= vac; R += vac
        return b, g, S, E, I, R
    return f

def testing_quarantine(start_day=70, reduction_days=2):
    # Increases recovery rate gamma to shorten infectious period
    def f(t,b,g,S,E,I,R):
        if t > start_day:
            g = 1.0 / (1.0/g - reduction_days)   # Effective gamma increase
        return b, g, S, E, I, R
    return f

def school_closure(start_day=70, duration=14, contact_factor=0.2):
    # Temporarily reduces beta during closure period
    return lambda t,b,g,S,E,I,R: (b*contact_factor if start_day <= t < start_day+du
                                  g, S, E, I, R)

# Fit the UVA data (Release #2) to obtain β,σ,γ for the first 70 days
df_uva = pd.read_csv("../Data/mystery_virus_daily_active_counts_RELEASE#2.csv",
                     parse_dates=['date'])
obs_uva = df_uva["active reported daily cases"].values
t_uva   = np.arange(len(obs_uva))

# UVA initial conditions
S0_uva = 17900
E0_uva = 0            # Exposed at start
I0_uva = 1
R0_uva = 0            # No recovered initially
R0_estimate = 1.24   # Provisional R0 from Release #1
N_uva  = S0_uva + E0_uva + I0_uva + R0_uva

# Perform grid search to fit parameters
best_beta, best_sigma, best_gamma, best_sse = grid_search_parameters(
    t_uva, N_uva, S0_uva, E0_uva, I0_uva, R0_uva, obs_uva
)
print(f"UVA fit: β={best_beta:.3f}, σ={best_sigma:.3f}, "
      f"γ={best_gamma:.3f}, SSE={best_sse:.0f}")

# Compare fitted R0 (β/γ) to initial estimate
effective_R0 = best_beta / best_gamma
print(f"Fitted R0 (β/γ): {effective_R0:.2f} (vs initial estimate {R0_estimate:.2f})

# VT setup: Simulate for Virginia Tech population
S0_vt = 38900           # VT student population size
E0_vt = E0_uva          # Same initial exposed as UVA
```

```python
I0_vt = 1
R0_vt = 0
N_vt  = S0_vt + E0_vt + I0_vt + R0_vt

horizon = 120           # Total simulation days

# Baseline simulation (no interventions)
S_base, E_base, I_base, R_base = simulate_piecewise(
    best_beta, best_sigma, best_gamma,
    S0_vt, E0_vt, I0_vt, R0_vt, N_vt,
    horizon, interventions=[]
)

# Function to compute peak and total infections in intervention period
def metrics(I_array, t0=70, t1=120):
    segment = I_array[t0:t1+1]
    return segment.max(), segment.sum()

base_peak, base_total = metrics(I_base)
print(f"baseline peak (70-120) = {base_peak:.0f}, total = {base_total:.0f}")

# Define intervention scenarios (starting at day 70)
scenarios = {
    "mask mandate"       : [mask_intervention()],  # Reduces transmission
    "vaccine campaign"   : [vacc_campaign()],      # One-time vaccination
    "vaccine rollout"    : [vacc_rollout()],       # Multi-day vaccination
    "testing + quarantine": [testing_quarantine()], # Increases recovery rate
    "school closure"     : [school_closure()],     # Temporary contact reduction
}

results = {}
# Simulate each scenario
for name, intervs in scenarios.items():
    S_i, E_i, I_i, R_i = simulate_piecewise(
        best_beta, best_sigma, best_gamma,
        S0_vt, E0_vt, I0_vt, R0_vt, N_vt,
        horizon, intervs
    )
    # Compute peak and total in intervention period
    peak, total = metrics(I_i)
    results[name] = (I_i, peak, total)
    print(f"{name:20s} peak={peak:.0f}, total={total:.0f}")

# Select best intervention based on lowest peak
best_name = min(results.items(), key=lambda kv: kv[1][1])[0]
print(f"\nrecommended intervention (lowest peak): {best_name}")

# Summarize impacts relative to baseline
print("\nIntervention impact summary (vs baseline):")
for name, (_, peak, total) in results.items():
    peak_red = (base_peak - peak) / base_peak * 100
    total_red = (base_total - total) / base_total * 100
    print(f"  {name:20s}: peak -{peak_red:5.1f}%, total -{total_red:5.1f}%")

# Plot the infected curves for all scenarios
plt.figure(figsize=(10,6))
```

```python
plt.plot(np.arange(horizon+1), I_base, label='baseline', lw=2)
for name,(I_i,_,_) in results.items():
    plt.plot(np.arange(horizon+1), I_i, label=name)
plt.axvline(70, color='k', ls='--', label='intervention start')
plt.xlabel('days')
plt.ylabel('infected')
plt.title('VT epidemic, days 0–120')
plt.legend()
plt.grid()
plt.show()
```

```
UVA fit: β=0.600, σ=0.200, γ=0.200, SSE=127546
Fitted R0 (β/γ): 3.00 (vs initial estimate 1.24)
baseline peak (70-120) = 5949, total = 145306
mask mandate         peak=4578, total=118458
vaccine campaign     peak=5564, total=135428
vaccine rollout      peak=5756, total=134171
testing + quarantine peak=4167, total=73404
school closure       peak=4251, total=113898

recommended intervention (lowest peak): testing + quarantine

Intervention impact summary (vs baseline):
  mask mandate        : peak - 23.1%, total - 18.5%
  vaccine campaign    : peak -  6.5%, total -  6.8%
  vaccine rollout     : peak -  3.3%, total -  7.7%
  testing + quarantine: peak - 30.0%, total - 49.5%
  school closure      : peak - 28.5%, total - 21.6%
```



Verify and validate your analysis:

We validated the SEIR fit (from Data Release #2) using **Data Release #3** as a held-out dataset. The validation metrics (RMSE, peak size error, and peak timing error) quantify how well our model generalizes beyond the data used for calibration.

Rubella (Rubivirus rubellae) fits the mystery virus:

- It is a 9.7 kb, positive-sense RNA virus with 50–70 nm enveloped particles. It is within the ~10 kb and 50-80 nm ranges given.
- The incubation period (14–18 days) explains the lag after midterm crowds, and the reported R_0 of 1.24 matches what we expect in a mostly vaccinated campus (effective R_e ≈1.2–1.3).
- Adults often get a rash and joint pain, and spread through respiratory droplets in crowded indoor spaces, which fits the library cluster scenario.

These points are backed by standard references (e.g. Lambert *et al.*, *Lancet* 2015) and by checking basic virology texts, so the analysis is consistent with published evidence.

Lambert N, Strebel P, Orenstein W, Icenogle J, Poland GA. Rubella. Lancet. 2015 Jun 6;385(9984):2297-307. doi: 10.1016/S0140-6736(14)60539-0. Epub 2015 Jan 8. PMID: 25576992; PMCID: PMC4514442.

Limitations and Future Work:

Error Calculation (Data Release #3 validation)

- Validation RMSE (Release #3): **674.5 active cases**
- Observed peak (Release #3): **3294 cases on day 82**
- Model predicted peak: **2738 cases on day 74**
- Peak size error: **–16.9%** (model underpredicts cases)
- Peak timing error: **–8 days** (model peaks earlier than observed)

These metrics show that the model captures the general shape of the outbreak but tends to underestimate the peak magnitude and occurs earlier than the observed data, which may reflect unmodeled changes in behavior or reporting dynamics after day 70.

Limitations

While the SEIR model provided information into the Rubella outbreak spread and intervention effectiveness, several limitations must be acknowledged. The model assumes homogeneous mixing within the population and constant transmission parameters, which may not accurately reflect real-world heterogeneity in contact patterns, immunity levels, or behavioral responses. The validation results (peak timing error of 8 days and a peak size error of ~17%) highlight remaining gaps between the model and the observed outbreak, which may stem from changes in behavior, reporting delays, or unmodeled factors like

asymptomatic transmission. The limited observational data also introduces uncertainty because cases may be underreported or reported with delays.

Future Work

Future research could incorporate more sophisticated modeling approaches. Implementing an age-structured or spatial SIR model would better capture population heterogeneity and localized transmission dynamics. Machine learning techniques for parameter optimization could reduce fitting errors. Collecting additional epidemiological data like contact tracing information would enable validation and refinement of the model. Extending the analysis to include agent-based simulations could explore the role of random events in outbreak control. Finally, integrating real-time data streams and adaptive intervention strategies would enhance the model's utility for ongoing public health decisions.

Conclusions and Key Findings

- **Model vs early estimate:** The SEIR fit implied an effective R_0 (β/γ) substantially higher than the early exponential estimate, demonstrating how early, incomplete data often understate transmissibility.
- **Validation results:** The model predicted the outbreak peak earlier (day 74 vs day 82) and slightly underpredicted peak magnitude (~17% lower). These errors highlight the importance of validating forecasts with new data and revising assumptions.
- **Intervention insights:** Among the tested interventions, testing + quarantine reduced peak infections the most (≈30% peak reduction, ≈50% total case reduction) by shortening the infectious period. Masking and vaccination still delivered benefits and would likely be combined in practice.
- **Takeaway for real-world decision-making:** Working with incomplete and evolving data requires communication of uncertainty and repeated validation. The approach shown here is a practical workflow for updating models as new data become available and using those models to compare intervention strategies.

Works Cited

Battisti, A. J., Yoder, J. D., Plevka, P., Winkler, D. C., Prasad, V. M., Kuhn, R. J., Frey, T. K., Steven, A. C., & Rossmann, M. G. (2012). Cryo-electron tomography of rubella virus. Journal of Virology, 86(20), 11078–11085. https://doi.org/10.1128/JVI.01390-12

Centers for Disease Control and Prevention. (2024). Rubella. In Epidemiology and prevention of vaccine-preventable diseases (The Pink Book). https://www.cdc.gov/pinkbook/hcp/table-of-contents/chapter-20-rubella.html

Hasan, S. S., Miller, A., Akanda, N., & Rossmann, M. G. (2018). Zika virus structure, maturation, and receptors. FEBS Letters, 592(16), 2713–2727. https://doi.org/10.1002/1873-3468.13139

Lambert, N., Strebel, P., Orenstein, W., Icenogle, J., & Poland, G. A. (2015). Rubella. The Lancet, 385(9984), 2297–2307. https://doi.org/10.1016/S0140-6736(14)60539-0

Mangla, A., & Hamad, H. (2024). Measles. In StatPearls. StatPearls Publishing. https://www.ncbi.nlm.nih.gov/books/NBK448068/

Qiu, J., Söderlund-Venermo, M., & Young, N. S. (2017). Human parvoviruses. Clinical Microbiology Reviews, 30(1), 43–113. https://doi.org/10.1128/CMR.00040-16

Shattock, A. J., Leuchter, R. K., Lodi, S., & Blumberg, S. (2022). Estimates of the basic reproduction number (R0) for rubella: A systematic review and meta-analysis. Vaccine, 40(10), 1450–1460. https://doi.org/10.1016/j.vaccine.2022.01.066

Swiss Institute of Bioinformatics. (n.d.). Rubivirus. ViralZone. https://viralzone.expasy.org/626